

Aula 3: npm, Node.js + Express.js e Rotas

Construindo as fundações do Back-end com JavaScript

Prof. Ronaldo Borges

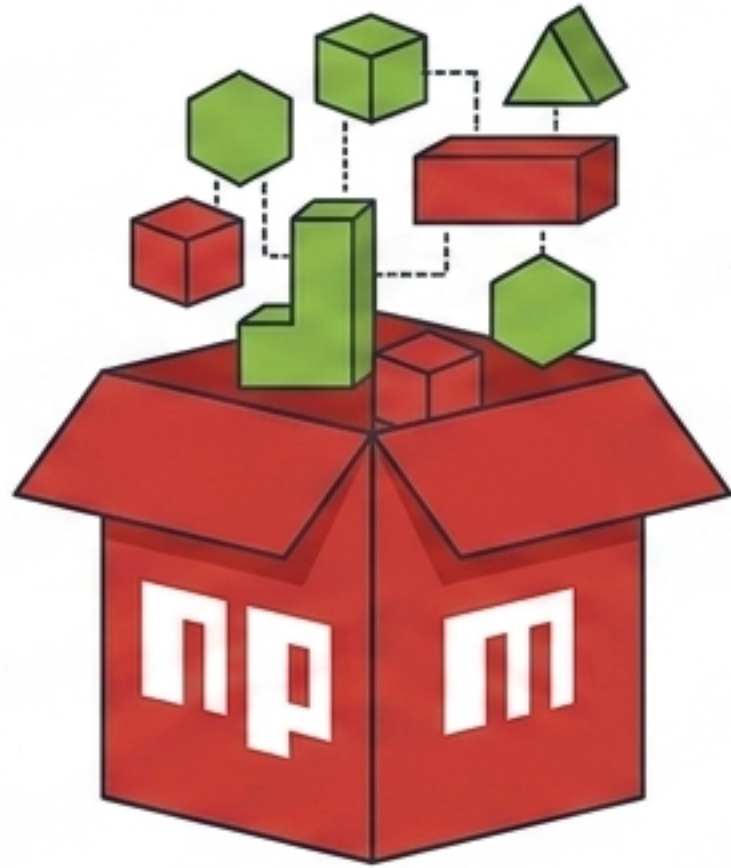
O Motor Back-end: Node.js (Recapitulando)

Onde o JavaScript ganha superpoderes.



npm: O Gerenciador de Pacotes

O ecossistema global do Node.



- Gerenciador padrão do ecossistema Node.
- Mantido pelo GitHub/Microsoft.
- **Facilita** compartilhamento de código de terceiros.

```
{  
  "name": "meu-projeto-backend",  
  "version": "1.0.0",  
  "scripts": {  
    "dev": "nodemon index.js"  
  },  
  "dependencies": {  
    "express": "^4.19.2"  
  }  
}
```

Atalhos de
execução

Materiais
instalados

Comandos Essenciais do npm

Como inicializar e gerenciar o projeto.



```
npm init -y
```

Cria o arquivo **package.json** automaticamente (**Yes to all**), definindo a identidade do projeto.



```
npm install express
```

Baixa o pacote do Express da nuvem, criando a pasta **node_modules** e atualizando as dependências.



```
npm run dev
```

Executa os atalhos de scripts criados por você no **package.json**.

A Chegada do Express.js

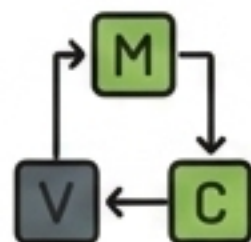
Minimalismo e Flexibilidade.

A filosofia do Express é fornecer ferramentas pequenas e robustas para servidores HTTP, sem esconder o Node.js.



Roteamento URL:

Direcionamento simples via métodos callbacks.



Fundações MVC & REST:

Ideal para construção de APIs RESTful e aplicações dinâmicas.



Middlewares:

Interceptadores que tratam requisições no meio do caminho.



Template Engines:

Integração transparente com renderizadores (como EJS) para construir views HTML.

Anatomia de um Servidor Express

Levantando a aplicação em 5 passos lógicos.

```
const express = require('express');  
const app = express();
```

```
app.get('/', (req, res) => {  
  res.send('Servidor rodando!');  
});
```

```
app.listen(8080, () => {  
  console.log('On-line!');  
});
```

1: Importa o módulo (O Material).

2: Inicializa a aplicação (A Fundação).

3: Define uma Rota base (A Porta de Entrada).

4: Responde ao cliente com texto/HTML.

5: Coloca o servidor em estado de escuta na porta 8080.

Entendendo o Roteamento (Rotas)

Direcionando o tráfego da aplicação.

Uma **rota** é sempre a combinação de **Caminho (URL)** + **Método HTTP**.



GET

Solicita e recupera dados
(Ex: Ler uma página).

POST

Envia novos dados ao servidor
(Ex: Salvar um formulário).

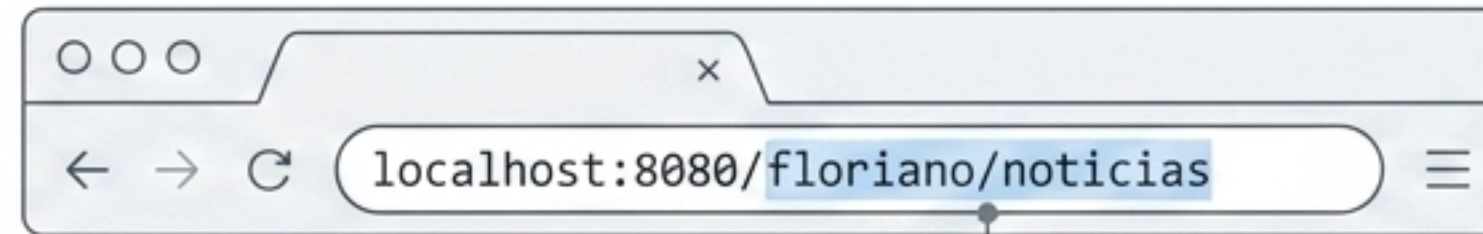
PUT

Atualiza/substitui dados
existentes.

DELETE

Remove uma informação.

Desenhando Caminhos Estáticos na Prática



```
// Rota Raiz (Home)
app.get('/', (req, res) => {
  res.send('Página Inicial');
});

// Rota Específica
app.get('/floriano', (req, res) => {
  res.send('Campus Floriano');
});

// Rota Aninhada
app.get('/floriano/noticias', (req, res) => {
  res.send('Notícias do Campus Floriano');
});
```


Matriz de Parâmetros

Como receber dados dinâmicos na URL.

Route Params

Propósito: Dados estruturais, identifica um recurso específico. Geralmente obrigatórios.

Sintaxe na URL:

```
/busca/node
```

Definição no Express:

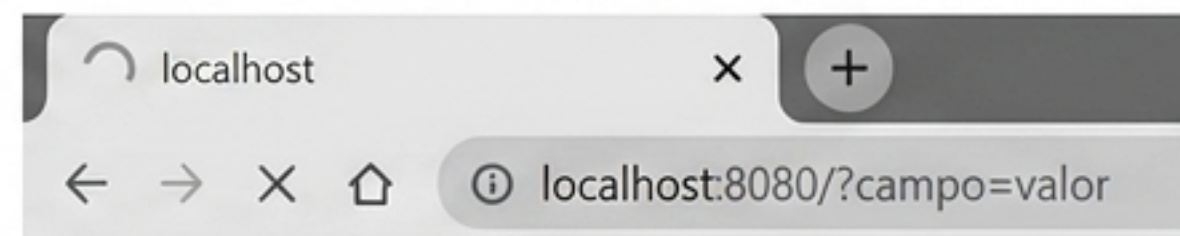
```
app.get('/busca/:conteudo', ...)
```

Captura no código:

```
req.params.conteudo
```

Query Params

Propósito: Filtros, buscas complexas ou modificadores. Opcionais.



Sintaxe na URL:

```
/busca?campo=valor
```

Definição no Express:

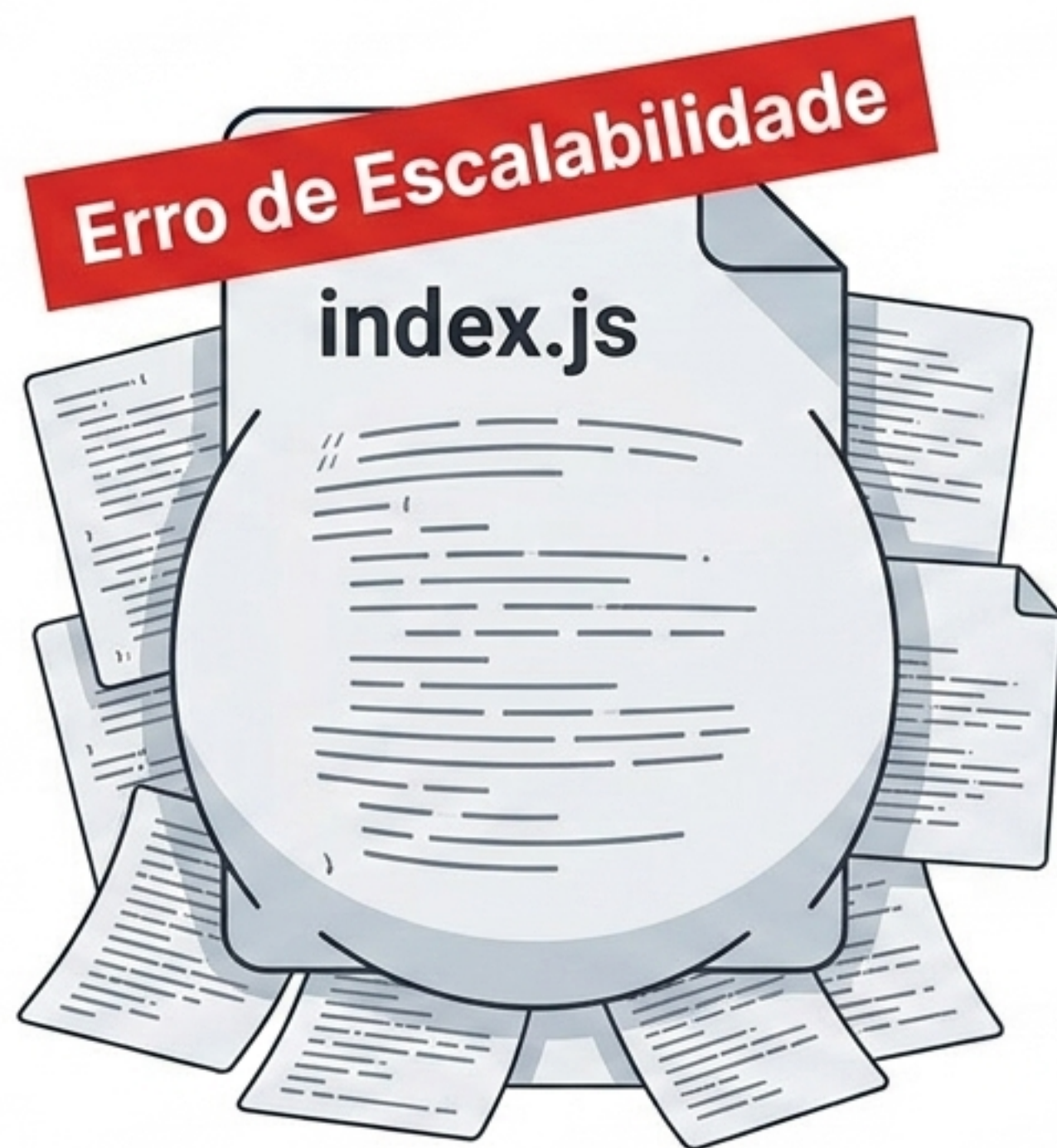
```
app.get('/busca', ...) (A rota ignora o query na declaração)
```

Captura no código:

```
req.query.campo
```


O Problema do Crescimento

Quando o index.js vira uma bagunça.



Colocar todas as rotas de usuários, produtos, pedidos e páginas em um **único arquivo de inicialização** (app.js ou index.js) torna o projeto **insustentável**.

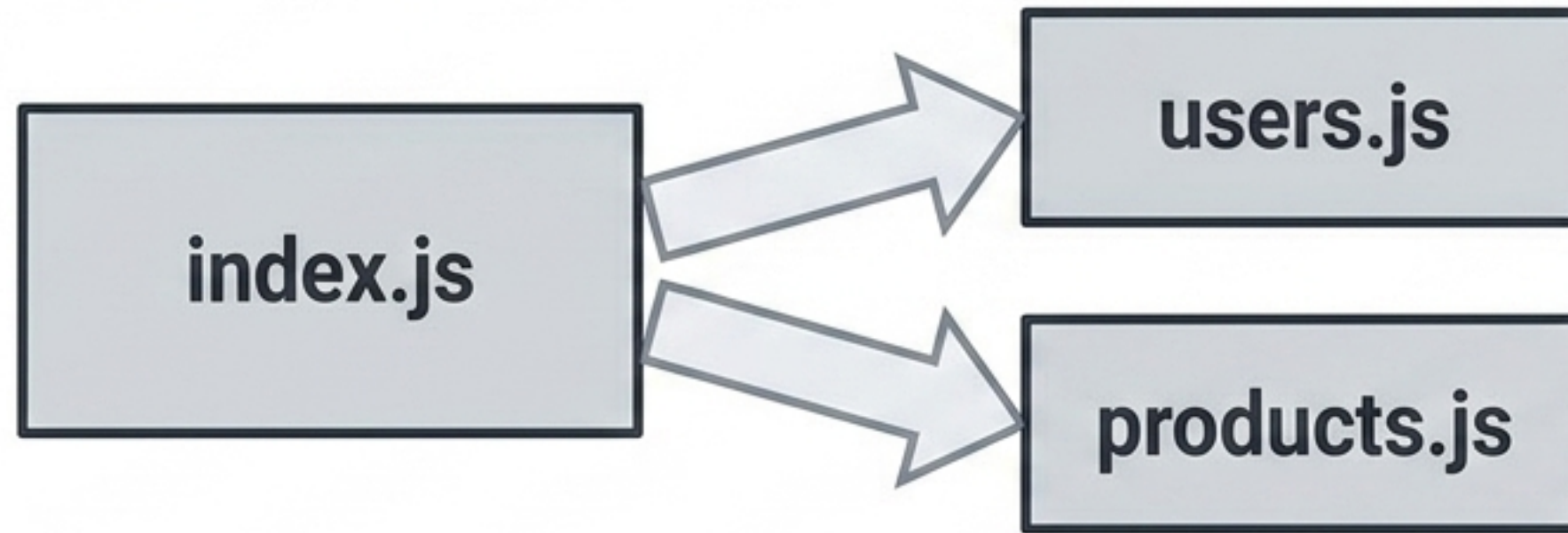
Consequências Práticas:

- ✗ Dificuldade extrema de manutenção.
- ✗ Conflitos constantes quando várias pessoas da equipe editam o mesmo arquivo.
- ✗ Lógica de negócio acoplada e impossível de testar isoladamente.

A Solução: Modularização

Separando Responsabilidades com express.Router()

O Router cria manipuladores de rotas modulares. Cada arquivo atua como um "mini-aplicativo" isolado que pode ser montado e reutilizado.



Arquivo de Rotas (novaRota.js)

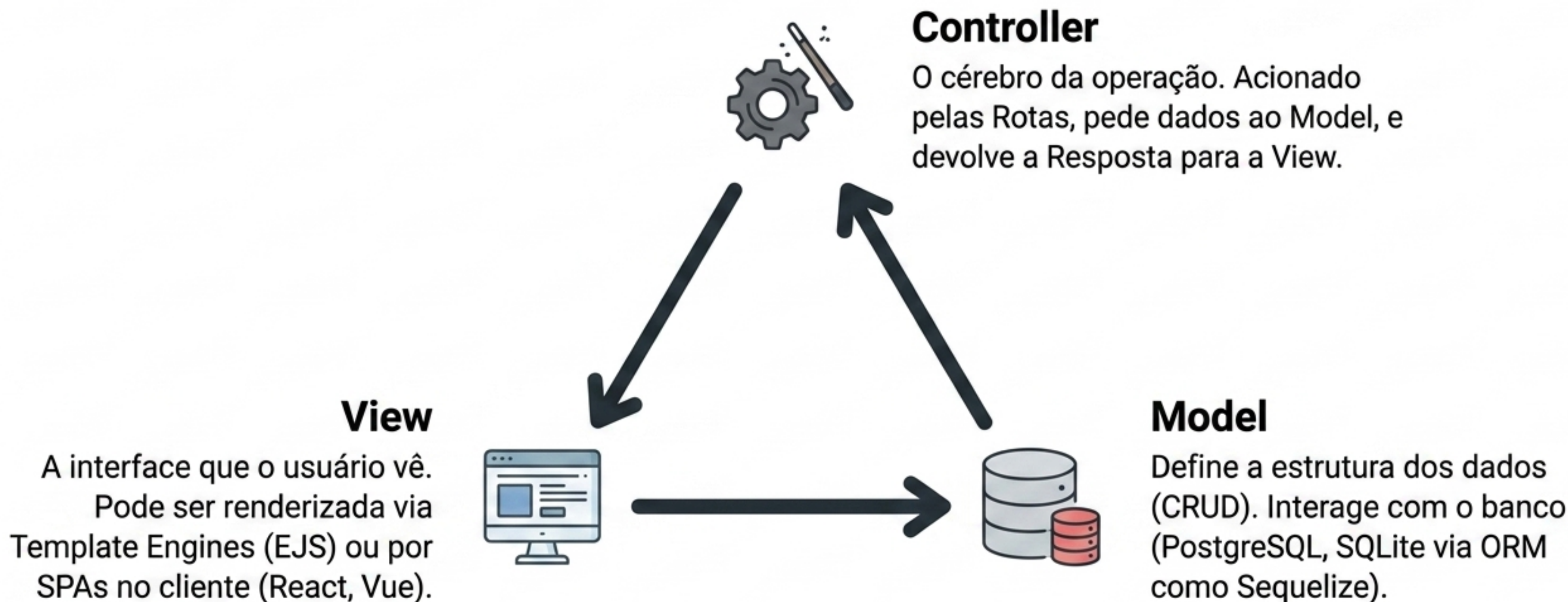
```
const router = express.Router();  
  
router.get('/novaRota', (req, res) => {...});  
  
module.exports = router;
```

Arquivo Principal (index.js)

```
const novaRota = require('./novaRota');  
  
app.use('/base', novaRota);
```

Arquitetura MVC (Model-View-Controller)

O padrão fundamental da Web.

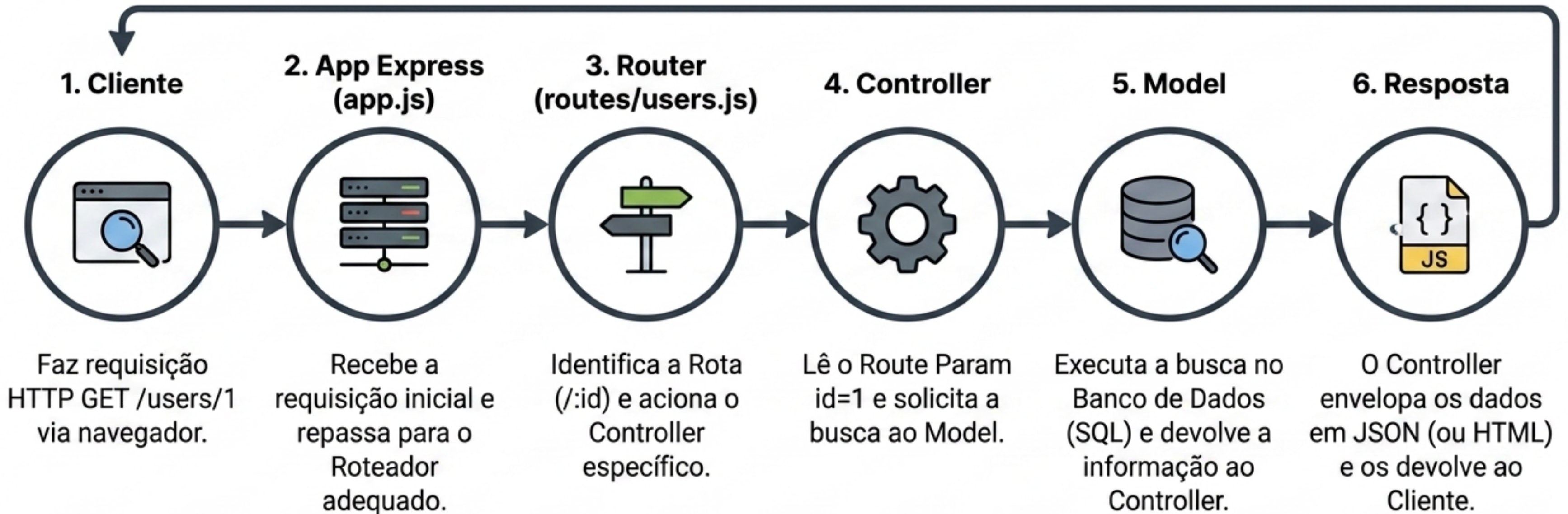


Estrutura de Diretórios Escalável

Prática MVC.

-  node_modules/ ----- (Módulos instalados via npm)
-  models/ ----- -> (Regras e Schemas de Banco de Dados)
-  controllers/ ----- -> (Lógica de negócio e ações das requisições)
-  routes/ ----- -> (Arquivos usando express.Router())
-  views/ ----- -> (Telas, e.g., arquivos .ejs)
-  app.js ----- (Ponto de entrada enxuto; apenas configurações globais)
-  package.json ----- (Identidade e dependências)

A Grande Imagem: Do Cliente ao Banco de Dados (e de volta)



Referências Bibliográficas

BROWN, Ethan. Programação web com Node e Express: Beneficiando-se da Stack JavaScript. São Paulo: Editora Novatec, 2020.

DOGLIO, Fernando. Pro REST API Development with Node.js. Nova York: Apress, 2015.

DUARTE, Luiz. Programação Web com Node.js: Completo, do Front-end ao Back-end. LuizTools. Edição do Kindle, [S.d.].

FLANAGAN, David. JavaScript: o guia definitivo. Porto Alegre: Bookman Editora, 2004.